

St. Francis Institute of Technology
Department of information technology
Academic year 2024-25
Class SEIT A/B SEM: IV

EXPERIMENT 1 : Python Basics

- 1. AIM :** To implement a Python program showing Identifiers, Keywords, Indention, Data Types, variables, operators, IDE's for python, installation
- 2. Objectives:** After performing this experiment, a student will be able to write a basic Python program using Python basic constructs
- 3. Prerequisite :** basic of programming, algorithms
- 4. Requirement :** PC, Python 3.9, Windows 10/ MacOS/ Linux , IDE
- 5. Pre-Experiment Exercise :**

Theory :

1) Numerical data types in python :

Python supports three kinds of numerical data.

- **Int:** Whole number worth can be any length, like numbers 10, 2, 29, - 20, - 150, and so on. An integer can be any length you want in Python. Its worth has a place with int.
- **Float:** Float stores drifting point numbers like 1.9, 9.902, 15.2, etc. It can be accurate to within 15 decimal places.
- **Complex:** An intricate number contains an arranged pair, i.e., $x + iy$, where x and y signify the genuine and non-existent parts separately. The complex numbers like $2.14j$, $2.0 + 2.3j$, etc.

2) Identifiers: Identifiers are things like variables. An Identifier is utilized to recognize the literals utilized in the program. The standards to name an identifier are given underneath.

- The variable's first character must be an underscore or alphabet (`_`).
- Every one of the characters with the exception of the main person might be a letter set of lower-case(a-z), capitalized (A-Z), highlight, or digit (0-9).
- White space and special characters (!, @, #, %, etc.) are not allowed in the identifier name. `^`, `&`, `*`.
- Identifier name should not be like any watchword characterized in the language. ○ Names of identifiers are case-sensitive; for instance, my name, and MyName isn't something very similar.

- Examples of valid identifiers: a123, _n, n_9, etc.
- Examples of invalid identifiers: 1a, n%4, n 9, etc.

3) **Keywords** : Python keywords are unique words reserved with defined meanings and functions that we can only apply for those functions

['False', 'None', 'True', 'and', 'as', 'assert', 'async', 'await', 'break', 'class', 'continue', 'def', 'del', 'elif', 'else', 'except', 'finally', 'for', 'from', 'global', 'if', 'import', 'in', 'is', 'lambda', 'nonlocal', 'not', 'or', 'pass', 'raise', 'return', 'try', 'while', 'with', 'yield']

4) **Operators** : Same as other languages, Python also has some operators, and these are given below -

- Arithmetic Operators - : +, -, /, %, *, **, //
- Comparison Operators : ==, !=, >=, >, <=, <
- Assignment Operators : =, +=, -=, /=, *=, %=, //=, **=, &=, |=, ^=, >>=, and <<=

Logical Operators : and, or, not

- Bitwise Operators bitwise OR (|), bitwise AND (&), bitwise XOR (^), negation (~), Left shift (<<), and Right shift (>>).
- Membership Operators : in, not in
- Identity Operators : is, is not

5) **Types of comments in python** :

- Single line comment #
- Using string literal ' , '

6) **IDE's for python** :

- IDLE (Integrated Development and Learning Environment) is a default editor that accompanies Python. This IDE is suitable for beginner-level developers. The IDLE tool can be used on Mac OS, Windows, and Linux. Price: Free
- PyCharm: PyCharm is a widely used Python IDE created by JetBrains. This IDE is suitable for professional developers and facilitates the development of large Python projects. Price: Freemium
- Visual Studio Code: Visual Studio Code is an open-source (and free) IDE created by Microsoft. It finds great use in Python development VS Code is lightweight and comes with powerful features that only some of the paid IDEs offer. Price: Free
- Sublime Text3
- Atom
- Jupyter
- Sypder
- Pydev etc.

7) **Installation** of python on windows PC : <https://www.geeksforgeeks.org/how-to-install-python-on-windows/>

8) Input and output in python :

- Python input() function is used to take user input. By default, it returns the user input in form of a string. Eg. *Ex: input("What is your name? ")*
To take integer input we will be using int() along with Python input()
Eg. num1 = int(input("Please Enter First Number: "))
- Python print() function prints the message to the screen or any other standard output device.

Syntax : `print(value(s), sep= ' ', end = '\n', file=file, flush=flush)`

Eg

```
name = "John"
age = 30
print("Name:", name)
print("Age:", age)
```

eg . `print("Hello, my name is", name, "and I am", age, "year")`

Eg:

```
a = 12
b = 12
c = 2022
print(a, b, c, sep="-")
```

You can format output in several ways:

1. **Using the format() method:**
2. **Using f-strings (Python 3.6+):**
3. **Using the % operator:**

4.

```
name = "Alice"
age = 30
print("Name: {}, Age: {}".format(name, age))
```
5.

```
name = "Alice"
age = 30
print(f"Name: {name}, Age: {age}")
```
6.

```
name = "Alice"
age = 30
print("Name: %s, Age: %d" % (name, age))
```

9) Indention :

In Python, indentation is used to define blocks of code. It tells the Python interpreter that a group of statements belongs to a specific block. All statements with the same level of indentation are considered part of the same block. Indentation is achieved using whitespace (spaces or tabs) at the beginning of each line.

Eg :

```
if 10 > 5:
```

```
    print("This is true!") #belongs to if block
```

```
    print("I am tab indentation") #belongs to if block
```

```
print("I have no indentation") #outside if block, part of main program
```

6. Laboratory exercise:

- a) Install Python on your PC and attach screenshots of the same
- b) Write a Python program to show use of all arithmetic operators
- c) Write a program to perform input and output in Python. Read marks of 3 students and print average mark of each student.

7. Post Experiment exercise:

7.1 Extended Theory

- a) Write various types of applications that can be created using python programming
- b) Difference between C/C++ and Python programming.
- c) Draw a table for Operator precedence and solve few expressions

7.2 Results/ observations / program output:

Present the program input/output results and comment on the same

7.3 Conclusion :

1. Write what was performed in the program (s).
2. What is the significance of program (s)?

8 References :

- 1) <https://www.javatpoint.com/python-applications>
<https://www.geeksforgeeks.org/python-output-using-print-function/?ref=lbp>

Python 3.12.1 (tags/v3.12.1:2305ca5, Dec 7 2023, 22:03:25) [MSC v.1937 64 bit (AMD64)] on win32

Type "help", "copyright", "credits" or "license()" for more information.

```
>>> a=10
>>> print(a)
10
>>> print(a,type(a))
10 <class 'int'>
>>> y=6.66
>>> print(y,type(y))
6.66 <class 'float'>
>>> x=False
>>> print(x,type(x))
False <class 'bool'>
>>> b=10
>>> a+b
20
>>> sum=y+a
>>> print(sum,type(sum))
16.66 <class 'float'>
>>> bin(a)
'0b1010'
>>> c=0b1010
>>> print(c)
10
>>> oct(25)
'0o31'
>>> bin
<built-in function bin>
>>> bin(0o31)
'0b11001'
>>> hex(0o31)
'0x19'
>>> bl=True
>>> print(bl,type(bl))
True <class 'bool'>
>>>
```

```

print(c,type(c))
10 <class 'int'>
c=2+4j
print
<built-in function print>
c=2+4j
print(c,type(c))
(2+4j) <class 'complex'>
c.real
2.0
c.imag
4.0
cl=3+6j
print(c+cl)
(5+10j)
b1=true

```

```

>>> s="python"
>>> s[0]
'p'
>>> s[10]
Traceback (most recent call last):
  File "<pyshell#63>", line 1, in <module>
    s[10]
IndexError: string index out of range
>>> s[-1]
'n'
>>> s[:]
'python'
>>> s[2:5]
'tho'
>>> s[5:2]
''
>>> s[-5:-2]
'yth'
>>> s[-1:2]
''
>>> s[-1:]
'n'
>>> s[2:-1]
'tho'
>>> #slicing with step s[Bindex : end index : step]
>>> s[0:5:1]
'pytho'
>>> s[0:5:2]
'pto'
>>> s[::-1]
'nohtyp'
>>> s[:]
'python'
>>> s[::1]
'python'
>>> s[::2]
'pto'
>>> s[::-1]
'nohtyp'

```

File Edit Format Run Options Window Help

```
#predefined methods in string class
s="welcome to python"
print("capitalize :",s.capitalize())
print("title :",s.title())
print("upppercase :",s.upper())
print("is uppercase :",s.isupper())
print("is lowercase:",s.islower())
print("is alphanum :",s.isalnum())
print("is digit :",s.isdigit())
print("is space :",s.isspace())
lst=s.split(" ")
print(lst,type(lst))
```

```
capitalize : Welcome to python
title : Welcome To Python
upppercase : WELCOME TO PYTHON
is uppercase : False
is lowercase: True
is alphanum : False
is digit : False
is space : False
['welcome', 'to', 'python'] <class 'list'>
```

```
>>>
```

IDLE Shell 3.11.5

File Edit Shell Debug Options Window Help

```
Python 3.11.5 (tags/v3.11.5:cce6ba9, Aug 24 2023, 14
:38:34) [MSC v.1936 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for m
ore information.
```

```
>>> s="Welcome To Python"
>>> s.swapcase()
'wELCOME tO pYTHON'
>>> s.split("")
```

```

>>> s.split(" ")
['Welcome', 'To', 'Python']
>>> lst=s.split(" ")
>>> print(Lst,type(lst))
['Welcome', 'To', 'Python'] <class 'list'>
>>> print(len(lst))
3
>>> print("#".join(lst))
Welcome#To#Python
>>> s.join(lst)
'WelcomeWelcome To PythonToWelcome To Python
Python'
>>>

```

#arithmetic operators

```

a=int(input("enter first number"))
b=int(input("enter second number"))
print("sum of{} and{} is {}".format(a,b,(a+b)))
print("diff of{} and{} is {}".format(a,b,(a-b)))
print("product of{} and{} is {}".format(a,b,(a*b)))
print("division of{} and{} is {}".format(a,b,(a/b)))
print("modulus of{} and{} is {}".format(a,b,(a%b)))

```

#comparison operators

```

print("{}<{} is {}".format(a,b,(a<b)))
print("{}>{} is {}".format(a,b,(a>b)))
print("{}=={} is {}".format(a,b,(a==b)))
print("{}!={} is {}".format(a,b,(a!=b)))

```

#bitwise operators

```

print("{}&{} is {}".format(a,b,(a&b)))
print("{}|{} is {}".format(a,b,(a|b)))
print("{}^{} is {}".format(a,b,(a^b)))
print("{}<<{} is {}".format(a,b,(a<<b)))
print("{}>>{} is {}".format(a,b,(a>>b)))
print("~{} is {}".format(a,(~a)))

```

#logical operators

```

a1=True
b1=False
print("{}and{} is {}".format(a1,b1,(a1 and b1)))
print("{}or{} is {}".format(a1,b1,(a1 or b1)))
print("not{} is {}".format(a1,(not a1)))

```

```
enter first number 20
enter second number 10
sum of 20 and 10 is 30
diff of 20 and 10 is 10
product of 20 and 10 is 200
division of 20 and 10 is 2.0
modulus of 20 and 10 is 0
20 < 10 is False
20 > 10 is True
20 == 10 is False
20 != 10 is True
20 & 10 is 0
20 | 10 is 30
20 ^ 10 is 30
20 << 10 is 20480
20 >> 10 is 0
~20 is -21
True and False is False
True or False is True
not True is False
```

```
s1 = str(input("enter the name of student"))
d1 = int(input("enter the marks of DSA"))
db1 = int(input("enter the marks of DBMS"))
m1 = int(input("enter the marks of MATHS"))
print("-----")
s2 = str(input("enter the name of student"))
d2 = int(input("enter the marks of DSA"))
db2 = int(input("enter the marks of DBMS"))
m2 = int(input("enter the marks of MATHS"))
print("-----")
s3 = str(input("enter the name of student"))
d3 = int(input("enter the marks of DSA"))
db3 = int(input("enter the marks of DBMS"))
m3 = int(input("enter the marks of MATHS"))
print("-----")
avg1 = (d1 + db1 + m1) / 3
avg2 = (d2 + db2 + m2) / 3
avg3 = (d3 + db3 + m3) / 3
print("The average marks of {} are {}".format(s1, avg1))
print("The average marks of {} are {}".format(s2, avg2))
print("The average marks of {} are {}".format(s3, avg3))
```

Installation


python
for
windows

Install Python 3.13.1 (64-bit)

Select **Install Now** to install Python with default settings, or choose **Customize** to enable or disable features.

 **Install Now**
C:\Users\chinm\AppData\Local\Programs\Python\Python313

Includes IDLE, pip and documentation
Creates shortcuts and file associations

 **Customize installation**
Choose location and features

☒ Use admin privileges when installing py.exe
☐ Add python.exe to PATH

Cancel


python
for
windows

Setup Progress

Installing:

Python 3.13.1 Test Suite (64-bit)

Cancel

